

اليوم 60 - JSON و XML و YAML

في عالم أتمتة الشبكات، نحتاج الأجهزة والبرامج إلى تبادل البيانات فيما بينها. سواء كان ذلك بين وحدة تحكم SDN وجهاز شبكة، أو بين أداة إدارة مثل Ansible وجهاز سيسكو، أو بين تطبيق و REST API، يجب أن تكون البيانات منظمة بطريقة مفهومة للجميع. هنا تأتي أهمية صيغ البيانات (Data Formats) مثل JSON و XML و YAML. هذه الصيغ الثلاث هي الأكثر استخداماً في أتمتة الشبكات وبرمجة التطبيقات (Web APIs). كل صيغة لها مميزاتها وعيوبها واستخداماتها المفضلة. في هذا الدرس، سنشرح كل صيغة بالتفصيل، ونقارن بينها، ونرى كيف تستخدم في سياق أتمتة الشبكات وسيسكو.

JSON (JavaScript Object Notation) هي أشهر صيغة لتبادل البيانات وأكثرها استخداماً في REST APIs. تم اشتقاق JSON من لغة JavaScript لكنها أصبحت مستقلة وتستخدم في جميع لغات البرمجة. تعتمد JSON على بنية أساسية بسيطة: الكائنات (Objects) محصورة بين { } وتحتوي على أزواج مفتاح-قيمة (Key-Value Pairs)، والمصفوفات (Arrays) محصورة بين [] وتحتوي على قائمة من العناصر. القيم يمكن أن تكون سلاسل نصية (Strings) وتحتها علامتي اقتباس، أو أرقاماً (Numbers)، أو قيماً منطقية (Booleans مثل true/false)، أو كائناً آخر، أو مصفوفة. JSON خفيف الوزن وسهل القراءة بالنسبة للألات، وسهل القراءة والكتابة بالنسبة للبشر أيضاً. في REST APIs الخاص بـ Cisco DNA Center، كل الاستجابات تعود بصيغة JSON. على سبيل المثال، عند طلب قائمة بالأجهزة عبر API، ستحصل على JSON يحتوي على مصفوفة من الكائنات، كل كائن يمثل جهازاً بمعلوماته (عنوان IP، اسم الجهاز، حالته، إلخ).

XML (eXtensible Markup Language) هي صيغة أقدم وأكثر رسمية من JSON. تعتمد XML على الوسوم (Tags) المشابهة لـ HTML لكنها قابلة للتوسيع (Extensible)، أي يمكنك تعريف وسومك الخاصة. XML أكثر تعقيداً من JSON ويحتوي على ميزات إضافية مثل السمات (Attributes) في الوسوم، والمساحات الاسمية (Namespaces)، و XSD (XML Schema Definition) لتحديد بنية المستند. XML أكثر طولاً (Verbose) من JSON، وهذا يعني أنه يستهلك عرض حزمة أكبر عند الإرسال. لكن XML له ميزة مهمة في أتمتة الشبكات: XML هو الأساس لبروتوكول NETCONF (Network Configuration Protocol) القياسي من IETF (RFC 6241). يستخدم XML لتبادل بيانات التكوين والبيانات التشغيلية بين مدير الشبكة والأجهزة. هذا يعني أن فهم XML ضروري للعمل مع NETCONF. على سبيل المثال، عند إرسال استعلام NETCONF لقراءة تكوين واجهة على جهاز سيسكو، يتم تغليف الطلب والاستجابة في XML.

YAML (YAML Ain't Markup Language) هي الصيغة الأحدث والأسهل قراءة للبشر من بين الثلاثة. تعتمد YAML على المسافة البادئة (Indentation) لتحديد بنية البيانات، بدلاً من الأقواس { } أو الوسوم (<>). هذا يجعل YAML أنظف وأسهل للقراءة والكتابة بشكل كبير. في YAML، يتم تعريف البيانات باستخدام أزواج مفتاح-قيمة (على غرار JSON)، والمسافة البادئة تحدد العلاقات الهرمية (التسلسل الهرمي). YAML تدعم أيضاً المصفوفات باستخدام شرطة (-) قبل كل عنصر. YAML هي الصيغة المستخدمة بشكل أساسي في Ansible Playbooks، وهي أيضاً صيغة شائعة في Kubernetes و Docker Compose والعديد من أدوات الأتمتة الحديثة. سهولة قراءة YAML تجعلها مثالية لملفات التكوين التي يكتبها البشر، حيث أن أي شخص يمكنه فهم بنية البيانات بمجرد النظر إليها.

نفس البيانات ممثلة في الصيغ الثلاث:

:JSON

```
{
  "device": {
    "hostname": "R1",
    "ip_address": "192.168.1.1",
    "interfaces": [
      {
        "name": "GigabitEthernet0/0",
        "ip_address": "10.0.0.1",
        "subnet_mask": "255.255.255.0",
        "status": "up"
      },
      {
        "name": "GigabitEthernet0/1",
        "ip_address": "10.0.1.1",
        "subnet_mask": "255.255.255.0",
        "status": "up"
      }
    ]
  }
}
```

:XML

```
<device>
  <hostname>R1</hostname>
  <ip_address>192.168.1.1</ip_address>
  <interfaces>
    <interface>
      <name>GigabitEthernet0/0</name>
      <ip_address>10.0.0.1</ip_address>
      <subnet_mask>255.255.255.0</subnet_mask>
      <status>up</status>
    </interface>
    <interface>
      <name>GigabitEthernet0/1</name>
      <ip_address>10.0.1.1</ip_address>
      <subnet_mask>255.255.255.0</subnet_mask>
      <status>up</status>
    </interface>
  </interfaces>
</device>
```

```
device:
  hostname: R1
  ip_address: 192.168.1.1
  interfaces:
    - name: GigabitEthernet0/0
      ip_address: 10.0.0.1
      subnet_mask: 255.255.255.0
      status: up
    - name: GigabitEthernet0/1
      ip_address: 10.0.1.1
      subnet_mask: 255.255.255.0
      status: up
```

دعونا نقارن بين الصيغ الثلاث بمزيد من التفصيل. JSON هي الأكثر استخداماً في REST APIs لأنها خفيفة، سهلة التحليل (Parsing) في جميع لغات البرمجة، وتدعم جميع هياكل البيانات الأساسية. جميع لغات البرمجة الحديثة تحتوي على مكتبات مدمجة أو خارجية لتحويل (Serialize/Deserialize) بيانات JSON إلى كائنات (Objects) في اللغة. في بايثون، مكتبة json هي جزء من المكتبة القياسية. XML هو الأكثر تعقيداً والأكثر طولاً، لكنه يظل مهماً في المؤسسات التي تستخدم NETCONF، وفي الأنظمة القديمة (Legacy Systems)، وفي الحالات التي تحتاج إلى تعريف رسمي ودقيق لبنية البيانات (باستخدام XML Schema Definition (XSD)). XML أيضاً يدعم التعليقات (Comments) التي لا يدعمها JSON رسمياً. YAML هو الأسهل قراءة للبشر ويستخدم بشكل أساسي في ملفات التهيئة (Configuration Files) و Playbooks في Ansible و Kubernetes. لكن YAML يمكن أن يكون حساساً جداً للمسافات البادئة (Spaces vs Tabs) ويمكن أن يؤدي خطأ بسيط في المسافة البادئة إلى أخطاء صعبة الاكتشاف.

في سياق أتمتة الشبكات مع سيسكو، كل صيغة تستخدم في مواقف مختلفة. REST APIs لـ Cisco DNA Center و Cisco Catalyst Center و Cisco Meraki تستخدم JSON لجميع طلبات واستجابات API. عندما تريد الحصول على قائمة الأجهزة من DNA Center، ترسل طلب GET إلى API وتتلقي استجابة بصيغة JSON. ثم تستخدم بايثون لتحليل الـ JSON واستخراج المعلومات التي تحتاجها. من ناحية أخرى، بروتوكول NETCONF يستخدم XML حصراً. إذا كنت تستخدم أداة مثل yangtool أو pyang للعمل مع YANG models (وهي نماذج بيانات تستخدم لوصف بيانات التكوين والحالة في NETCONF)، فستتعامل مع XML باستمرار. أما YAML، فتستخدمها في Ansible Playbooks لوصف المهام التي تريد تنفيذها على أجهزة الشبكة. كل Ansible Playbook هو ملف YAML يصف "hosts" و "tasks" و "variables".

متى تستخدم كل صيغة؟ القاعدة العامة بسيطة: استخدم JSON عندما تتعامل مع REST APIs أو عندما تحتاج إلى تبادل بيانات بين تطبيقات مختلفة (لأن JSON هو معيار الصناعة). استخدم XML عندما تتعامل مع NETCONF أو مع أنظمة قديمة أو عندما تحتاج إلى تعريف رسمي لبنية البيانات عبر XSD. استخدم YAML عندما تكتب ملفات تهيئة للبشر (مثل Ansible Playbooks أو Docker Compose) أو عندما تحتاج إلى صيغة نظيفة وقابلة للقراءة. في كثير من الأحيان، ستستخدم أكثر من صيغة في نفس المشروع. على سبيل المثال، قد تكتب (Ansible Playbook (YAML يستدعي (JSON REST API) لاسترجاع بيانات من DNA Center، أو يرسل استعلام XML (NETCONF) لجلب تكوين جهاز. المهم هو أن تفهم كل صيغة ونقاط قوتها لتتمكن من استخدامها بشكل مناسب في سياقها الصحيح.

في الختام، إتقان JSON و XML و YAML هو مهارة أساسية لأي مهندس شبكات يريد العمل في مجال أتمتة الشبكات. هذه الصيغ الثلاث هي لغة التواصل في عالم الأتمتة. بدون فهمها، لا يمكنك قراءة استجابة API، أو كتابة Ansible Playbook، أو فهم استعلام NETCONF. ابدأ بحفظ البنية الأساسية لكل صيغة، وتدريب على تحويل البيانات بينها (مثلاً، خذ بيانات JSON وحولها يدوياً إلى YAML والعكس). هناك أدوات عديدة على الإنترنت تساعد في التحويل بين الصيغ، لكن الفهم اليدوي ضروري لتتمكن من قراءة وكتابة هذه الصيغ بطلاقة. تذكر: JSON للآلات، YAML للبشر، XML للمؤسسات الرسمية. كل منها له مكانه في مجموعة أدوات مهندس الشبكات الحديث.